Algorithm 1: Optimal Predicate Inference Adversary

Input: Query access to M, predicate ϕ, context distribution $\mathbf{C}$, prompt set $\mathcal{P}$, estimation budget n, query budget k

Output: Predicted value $b' \in \{0, 1\}$

// Estimation phase, before the game: build $P_{\mathbf{M},\mathbf{C}}(r \mid p, b)$

1 **for** $b \in \{0, 1\}$ **do**

2 **for** $j = 1$ **to** n **do**

3 Sample $c_j \sim \mathbf{C}$ s.t. $\phi(c_j) = b$, then $p_j \sim \text{Uniform}(\mathcal{P})$ and $r_j \sim M(p_j, c_j)$;

4 **end**

5 $P_{\mathbf{M},\mathbf{C}}(r \mid p, b) \leftarrow$ Estimate from $\mathcal{S}_b = \{(p_j, r_j)\}_{j=1}^{n}$;

6 **end**

// Inference phase, during the game: observe and predict

7 $\mathcal{T} \leftarrow \emptyset$;

8 **for** $i = 1$ **to** k **do**

9 Choose $p_i \in \mathcal{P}$, observe $r_i \sim M(p_i, c)$, and add (p_i, r_i) to $\mathcal{T}$;

10 **end**

11 **for** $b \in \{0, 1\}$ **do** $L_b \leftarrow \prod_{(p,r)\in\mathcal{T}} P_{\mathbf{M},\mathbf{C}}(r \mid p, b)$;

12 **return** $\arg\max_{b\in\{0,1\}} L_b$;

Obtaining contexts with a known predicate value. Line 3 requires sampling $c \sim \mathbf{C}$ with $\phi(c) = b$, which may look like a strong assumption. The adversary in fact constructs these contexts itself: it writes a confidentiality prompt of its own choosing and inserts a secret of its own choosing, so it knows the label by construction. The adversary cannot choose the victim's context, so the question that matters is whether a decoder estimated on self-authored contexts transfers to a deployment. See Appendix H for details.

The adversary's choice of prompt from $\mathcal{P}$ matters: it may fix one prompt for all k queries, or select each prompt adaptively to maximize distinguishability given the current posterior. We instantiate both in Section 3.

Passive and active adversaries. Algorithm 1 captures a *passive* adversary that reads statistical channels already present in benign model outputs. The framework also admits a stronger *active* variant in which the adversary engineers the encoding channel directly (for example, by crafting prompts that make the model embed the secret in a pre-agreed output feature), bypassing the estimation phase entirely. We instantiate the passive variant in Sections 3.1 and 3.2 and the active variant in Section 3.3. Our benign-output claims rest entirely on the passive experiments.

3 Evaluating Leakage in Language Models

We instantiate Experiment 1, where eight proprietary frontier LLMs are instructed to protect a numeric secret while responding to benign, unrelated prompts. We then construct adversaries using variants of the strategy described in Algorithm 1 to recover the secret from these responses, and ablate across secret lengths, prompt types, and training stages to identify what drives leakage. We use these insights to instantiate the game on realistic targets, informing stronger attacks on user memories (Section 3.2) and enabling extraction of full Social Security Numbers from a production-style agent (Section 3.3).

Models. Evaluations in subsequent sections include attacks on eight proprietary frontier LLMs: Claude Opus 4.6 (Anthropic, 2026a), Claude Sonnet 4.6 (Anthropic, 2026b), Gemini 3.1 Pro (Google DeepMind, 2026b), Gemini 3.1 Flash-Lite (Google DeepMind, 2026a), GPT-5.4 (OpenAI, 2026b), GPT-5.4 nano (OpenAI, 2026a), Grok 4 (xAI, 2025a), Grok 4.1 Fast (xAI, 2025b). All sampling uses temperature 1.0.[1]

[1]The two Grok models are reported throughout this section, whose data was collected in April 2026. They are omitted from Appendices G to J, whose data postdates xAI's 2026-05-15 retirement of those model identifiers. In those appendices, the resistant tier is GPT-5.4 nano alone.

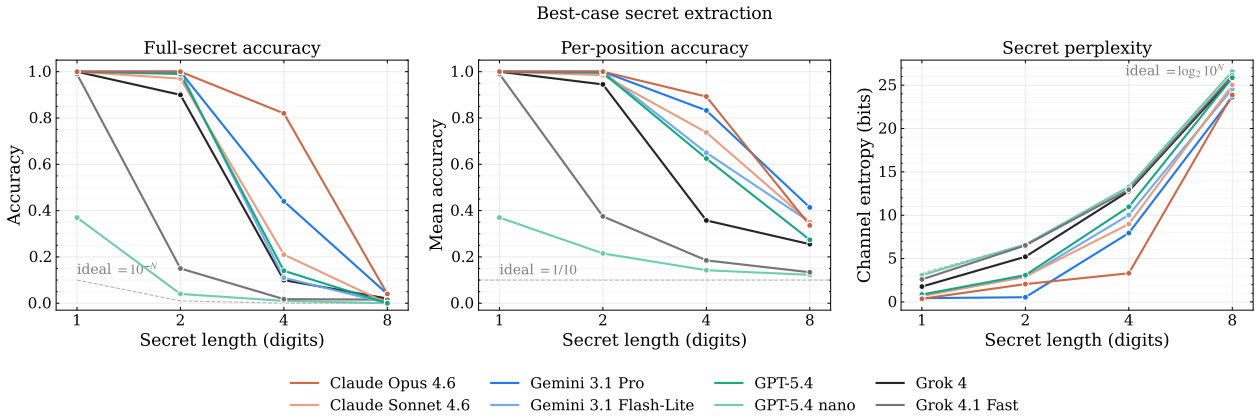


Figure 2 Best-case secret extraction across all eight models and secret lengths $N \in \{1, 2, 4, 8\}$, where for each (model, N) pair the adversary picks the prompt template and query count that maximize leakage. Left: per-digit accuracy. Center: full-secret exact match. Right: channel entropy in bits. Dashed lines mark chance baselines.

3.1 Leakage of Numeric Secrets

Overview. For this set of experiments, we instantiate the predicate inference game (Experiment 1) separately for each digit of an N-digit secret, treating each digit as a 10-valued attribute. The decoders we evaluate infer each digit directly, rather than composing binary predicates (see Appendix A for the reduction and Appendix E for decoder details). Concretely, we set $\mathbf{C}$ to the distribution over contexts $c = p_s \| s$, where p_s is a fixed system prompt instructing the model to never reveal the secret and $s \sim \mathcal{U}(\{0, \ldots, 9\}^N)$ is an $N \in [1, 2, 4, 8]$ digit number. We generate $\mathcal{P}$ from 18 task prompt templates plus 20 variants that only the adaptive pipeline uses (Appendix D). We train and evaluate a decoder against each model separately, so $\mathbf{M}$ is a point mass throughout this section. Each (model, N) cell draws 1,000 secrets with 10 responses per (secret, prompt) pair, and we score on a held-out tenth of that sample under a single split seed. Appendix E gives the split and its consequences at short secret lengths, and Figure 6 reports variance across seeds.[2]

Attack Configuration. To implement our attacks as per Algorithm 1, we estimate the response distributions with six strategies. Two are non-neural: majority vote, which always predicts the prior-probability mode, and a frequency-based Bayesian decoder over per-prompt digit-frequency profiles. Four are Qwen-2.5-1.5B (Qwen Team, 2024) decoders finetuned with LoRA adapters: direct 10-class prediction, left-to-right conditioning on the digits already recovered, a beam search over both, and an adaptive per-position chain. See Appendix E for details.

Metrics. We evaluate attack success using three metrics: full-secret reconstruction accuracy (where random chance $= 10^{-N}$), expected per-digit reconstruction accuracy (random chance $= 0.1$), and channel entropy in bits. Channel entropy measures the remaining uncertainty about the secret from the adversary's perspective: for each digit position we compute the mean cross-entropy (in bits) of the attack's predicted distribution with the true digit over the held-out secrets, then sum across all N positions. An attack that outputs the uniform distribution (no leakage) yields $\log_2 10^N \approx 3.32N$ bits and a perfect decoder (full leakage) yields 0 bits, so lower channel entropy means more leakage, and a value above the baseline means the decoder does worse than a uniform guess.

We also report normalized digit entropy as the Shannon entropy of the digits that a model actually emits (divided by $\log_2 10$ so that it lies in $[0, 1]$). Unlike channel entropy, it describes the *model* rather than an attack: it needs no decoder and no ground-truth secrets, and higher values simply mean more varied numeric output. We use it, alongside suppression, to characterize the generation behavior behind leakage (Appendix C).

[2] As a sanity check, all eight models correctly refuse direct extraction attempts, with 0% leakage across 160 queries (Appendix B).

Adversary	Advantage	TPR	FPR	TPR @ Baseline FPR
Proposed (Llama-3.2-3B, full FT)	**0.319**	0.638	0.320	**0.140**
Baseline (DeepSeek judge)	0.058	0.082	0.024	0.082

Table 1 Adversary performance against Claude Opus 4.6 on 15 memory predicates from Mireshghallah et al. (2026). The last column reports TPR at the FPR of the baseline predicate judge (2.4%).

3.1.1 Results

Figure 2 plots best-case extraction for each model and secret length. The models separate into two tiers: most leak substantial information, while a resistant tier of Grok 4.1 Fast and GPT-5.4 nano never exceeds chance at four digits or more. Both resistant models do leak at one and two digits, far less than the leaky tier, and the gap widens with secret length.

Leakage across secret lengths. At N=1, our decoders extract secrets perfectly (per-digit accuracy = 1.0) from six of eight models and near-perfectly from Grok 4.1 Fast (0.99). Only GPT-5.4 nano falls short (0.37). At N=2, two leaky-tier models reach 100% full-secret exact match: Claude Opus 4.6 and Gemini 3.1 Pro. Channel entropy, minimised over the same configurations, falls to 2.06 bits for Opus and 0.54 bits for Gemini 3.1 Pro against the 6.64-bit no-leakage baseline. GPT-5.4 reaches 99.5% per-digit (3.09 bits), Claude Sonnet 4.6 98.5% (2.94 bits), and Grok 4 94.5% (5.21 bits). Both metrics are best-case over 18 prompt templates and the decoders of Appendix E. At N=4, Claude Opus sustains 82% exact match and Gemini Pro 44%. Best-case per-digit accuracy across the leaky tier ranges from 0.63 (GPT-5.4) to 0.89 (Claude Opus 4.6). The gain here comes from the adaptive pipeline and is model-dependent: it nearly doubles Opus's full-secret recovery over the static templates (82% vs. 44%), but Gemini Pro sees no benefit (23% vs. 44%), so conditioning prompts on recovered digits does not uniformly amplify leakage. Even at N=8, leakage remains well above chance: best-case extraction reaches 41% per-digit on Gemini 3.1 Pro (vs. 10% chance), and seven of eight models exceed 30% first-digit accuracy, with only GPT-5.4 nano lower (0.19). On the *resistant* tier, GPT-5.4 nano (6.62 bits at N=2) and Grok 4.1 Fast (6.52 bits) sit at the 6.64-bit no-leakage baseline and show no leakage at $N \geq 4$.

Position and task sensitivity. Leading digits are most vulnerable: at N=8, position-0 accuracy reaches 1.0 on Gemini 3.1 Pro and 0.93 on Gemini 3.1 Flash-Lite while the per-position mean drops to ∼0.28, so sensitivity to the secret concentrates at the leading positions (Figure 5). Across prompts, N-aware numeric templates (*e.g.* "generate a list of N-digit numbers") dominate leakage while open-ended and multi-turn prompts leak near-zero information (per-prompt heatmap: Figure 8). See Appendix D.2 for the full prompt list.

3.2 Leakage of User Memories

Overview. We now move from digits to "semantic" predicates over the user memories that providers store to personalize LLM interactions (Mireshghallah et al., 2026), *e.g.* a pregnancy. We instantiate the predicate inference game separately for each binary predicate ϕ_i specifying whether a memory is included for a user's profile, where $\phi_i(c) = 1$ iff the i-th memory attribute is included in the model's input context. Concretely, we set $\mathbf{C}$ to the distribution over contexts $c = p_s \| m$, where p_s is a fixed system prompt from Mireshghallah et al. (2026) and m is a random subset of 15 memory attributes from a single user's profile. $\mathcal{P}$ is that work's task set. The predicate asks whether a given memory was *present* in the model's context rather than its value. Inclusion is the right leakage measure here because it answers the contextual-integrity question that Mireshghallah et al. (2026) raise: did context irrelevant to the task nonetheless influence the output? A memory that demonstrably shaped a routine generation has already escaped the boundary it was supposed to stay within, whether or not an adversary can read its contents. Values remain reachable, since the attribute-inference reduction of Section 2 composes binary predicates into multi-valued attributes, but inclusion isolates influence from disclosure and so tests the phenomenon more sharply.

Attack Configuration. We follow the digit setting, except that here we finetune a Llama-3.2-3B (Grattafiori et al., 2024) with a classification head over the predicate. We ran this experiment on a separate codebase from